Foundation models are defined as the base model trained on large-scale data in a self-supervised or semi-supervised manner that can be adopted for several other downstream tasks

Single Task Model

Trained for one specific task



Input Image

Model
(Trained for one task)

Output

Dog
(✓)

What it can do

Performs well on the specific task it was trained for.

Limitations

- ✗ Needs a separate model for each task
- ✗ Cannot understand language or generalize beyond its training objective
- ✗ Limited adaptability to new tasks or domains

Vision-Language Foundation Model

Trained on images and text at scale



Input Image

Vision-Language Foundation Model
(Pretrained on Image + Text)

Many Capabilities

- **Captioning** "A dog sitting on the grass."
- **Image Retrieval** Find similar images
- **Visual QA** "What is the animal?" → "Dog
- **Zero-shot Classification** Dog (✓)
- **Segmentation** (Dog mask)

What it can do

Handles many tasks: recognition, captioning, retrieval, VQA, segmentation, and more.

Understands both visual content and natural language.

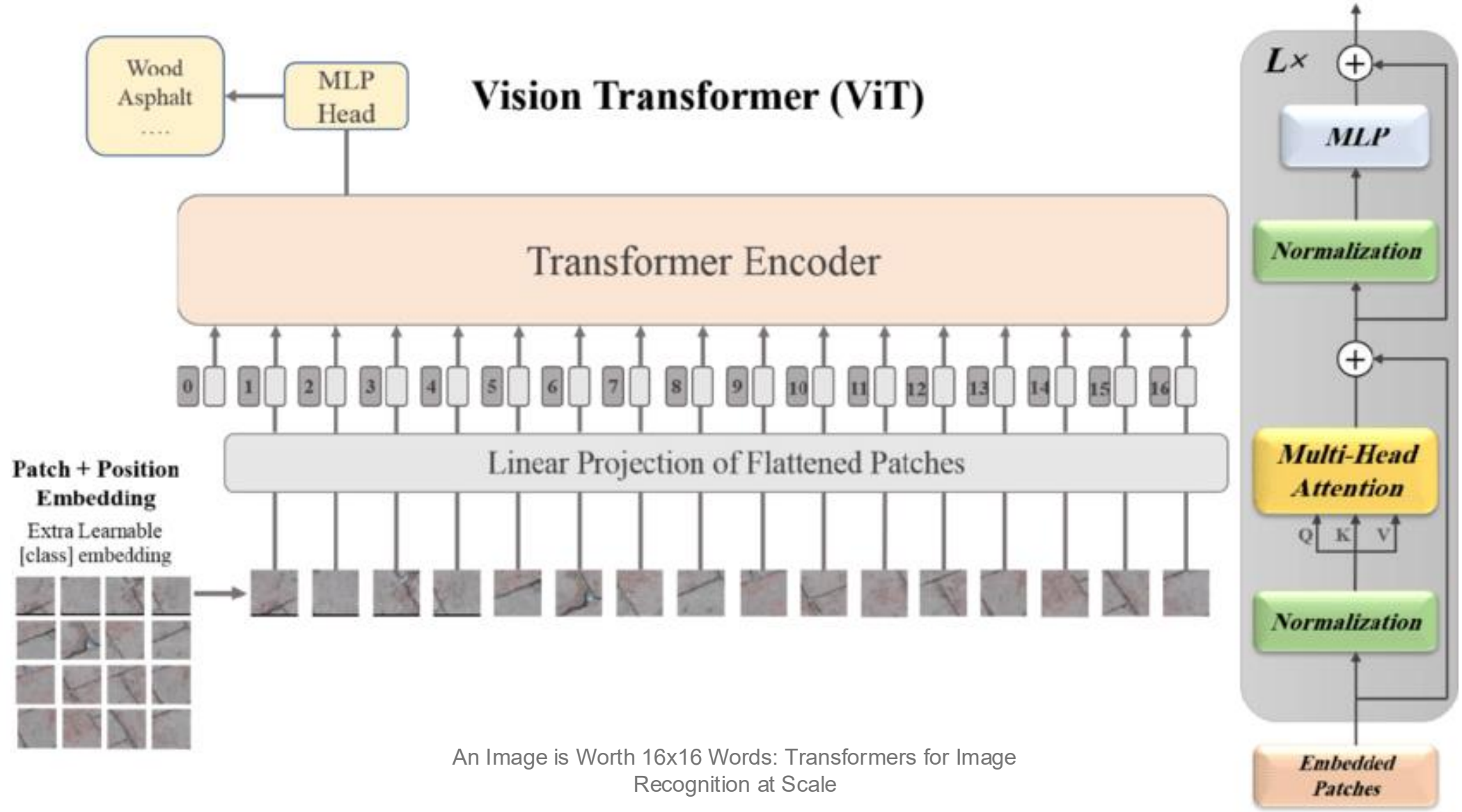
Benefits

- ✓ One model, many tasks
- ✓ Better generalization to new tasks and domains
- ✓ Leverages text to provide rich, interpretable understanding

Vision-only foundation models

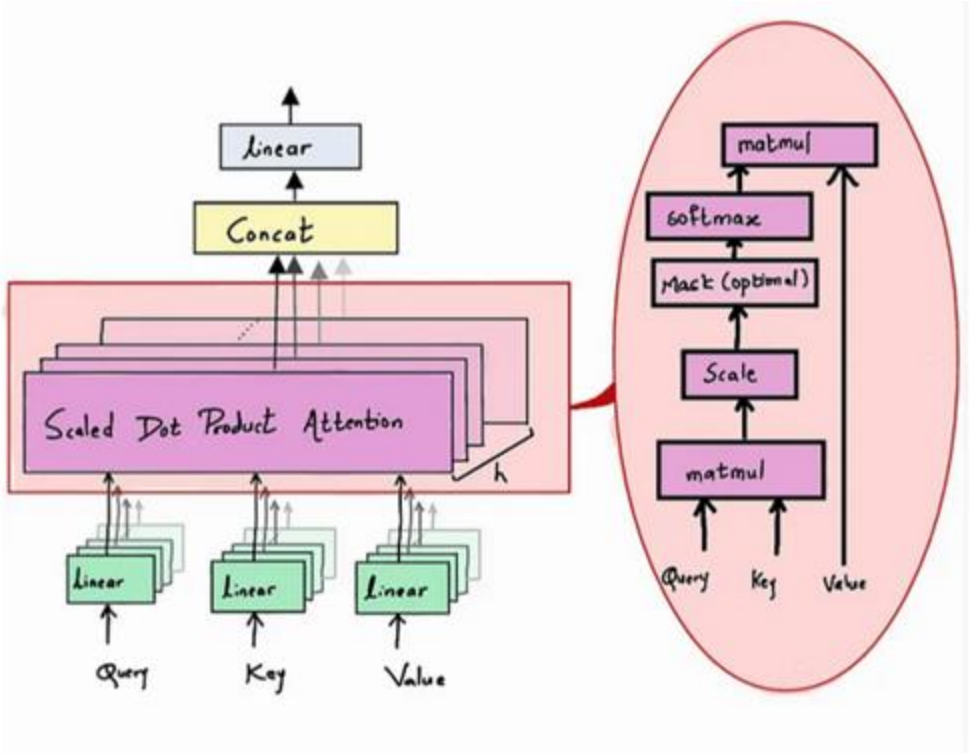
- **ViT** – transformer backbone for image understanding and recognition
- **MAE** – self-supervised masked image modeling for transferable visual features
- **DINOv2 / DINOv3** – self-supervised general-purpose visual representation models trained without text supervision
- **SAM / SAM 2** – promptable segmentation foundation models for images, with SAM 2 extending to videos as well
- **ConvNeXt** – a strong modern ConvNet often discussed as a scalable vision backbone alongside transformer families

Vision Transformers



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Self attention – encoding long-range contexts



Given a sequence of token embeddings $X \in \mathbb{R}^{n \times d}$, we project them into:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

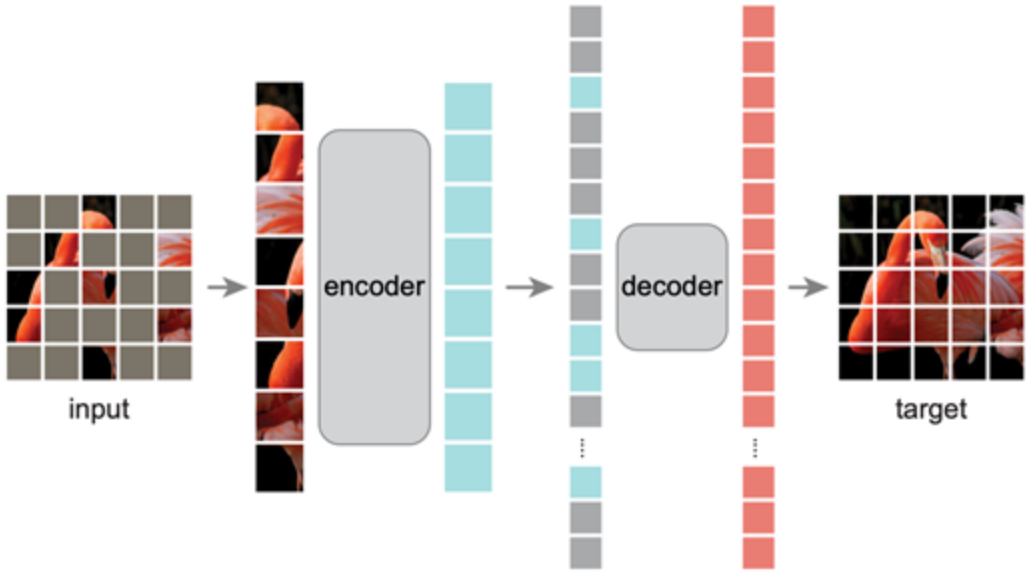
Then attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

The output is a matrix $A \in \mathbb{R}^{n \times d_v}$, where:

- Each row $A_i \in \mathbb{R}^{d_v}$ is a **contextualized representation** of token i
- It is computed as a **weighted sum of all value vectors** V_j, where weights depend on the similarity between Q_i and K_j

Masked autoencoding



Masked autoencoding is a self-supervised learning strategy: a large fraction of image patches is hidden, and the model learns to reconstruct the missing content from the visible patches.

It forces semantic understanding: since most of the image is masked, the encoder must capture global structure and meaningful visual patterns rather than rely only on local texture cues.

MAE is efficient and scalable: only visible patches are processed by the encoder, reducing computation during pretraining while still learning strong transferable representations.

The learned features generalize well: after pretraining, the model can be fine-tuned for downstream tasks such as classification, detection, and segmentation.

before pairing images with lanMAE is important for VLMs because it gives the **vision encoder**
a strong, scalable way to learn visual representations
guage.

Distillation with No Labels (DINO)

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

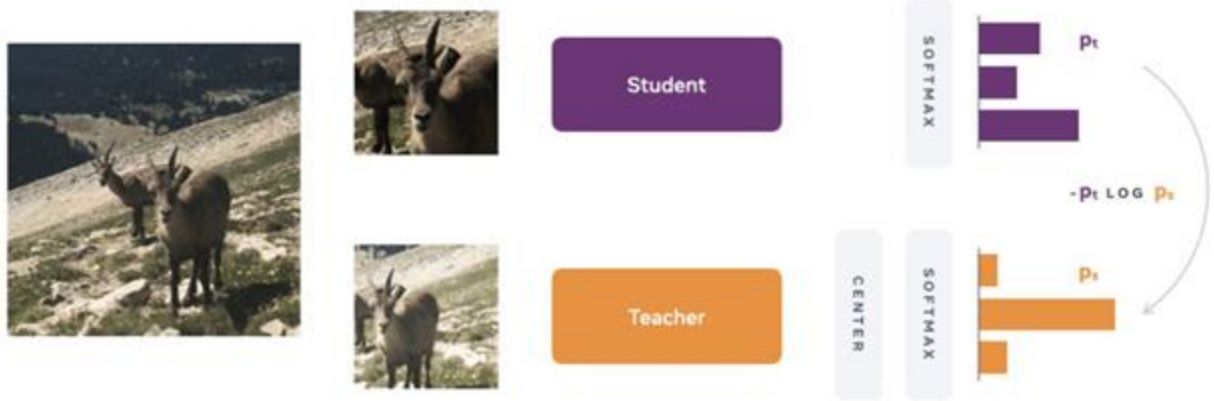
```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

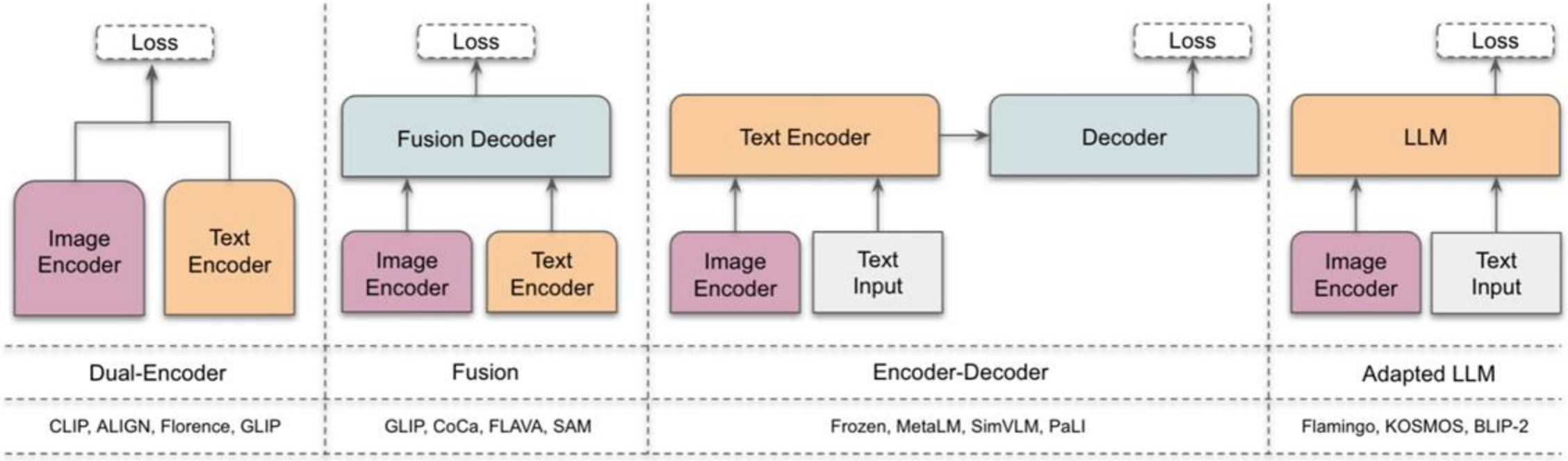
    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```



Emerging Properties in Self-Supervised Vision Transformers



Architectures for vision-language FM

- **Representation challenge**: map vision and language into compatible feature spaces.
- **Alignment challenge**: ensure that corresponding visual and textual semantics are associated.
- **Reasoning/generation challenge**: use the combined information to produce a useful output.

Dual encoder model

These learn separate encoders:

$$z_v = f_v(x_v), \qquad z_t = f_t(x_t),$$

and optimize them so matched image-text pairs have high similarity.

The similarity is often cosine similarity:

$$s(x_v, x_t) = \frac{z_v^\top z_t}{\|z_v\| \, \|z_t\|}.$$

This family is excellent for retrieval, zero-shot classification, and efficient indexing.

Cross-attention model

These allow image and text tokens to interact inside the Transformer through cross-attention or co-attention. They are stronger for tasks like VQA, grounding, and multimodal understanding because token-level interactions are explicit.

If V is a set of visual tokens and T is a set of text tokens, cross-attention computes something like

$$\mathrm{Attn}(Q_T, K_V, V_V) = \mathrm{softmax}\left(\frac{Q_T K_V^\top}{\sqrt{d}}\right) V_V,$$

so text queries can attend to visual keys and values.

Encoder to LLM connector model

These are currently among the most influential in open multimodal LMs. A pretrained vision encoder produces visual tokens or features, then a lightweight connector maps them into the token space expected by an LLM.

Typical pipeline:

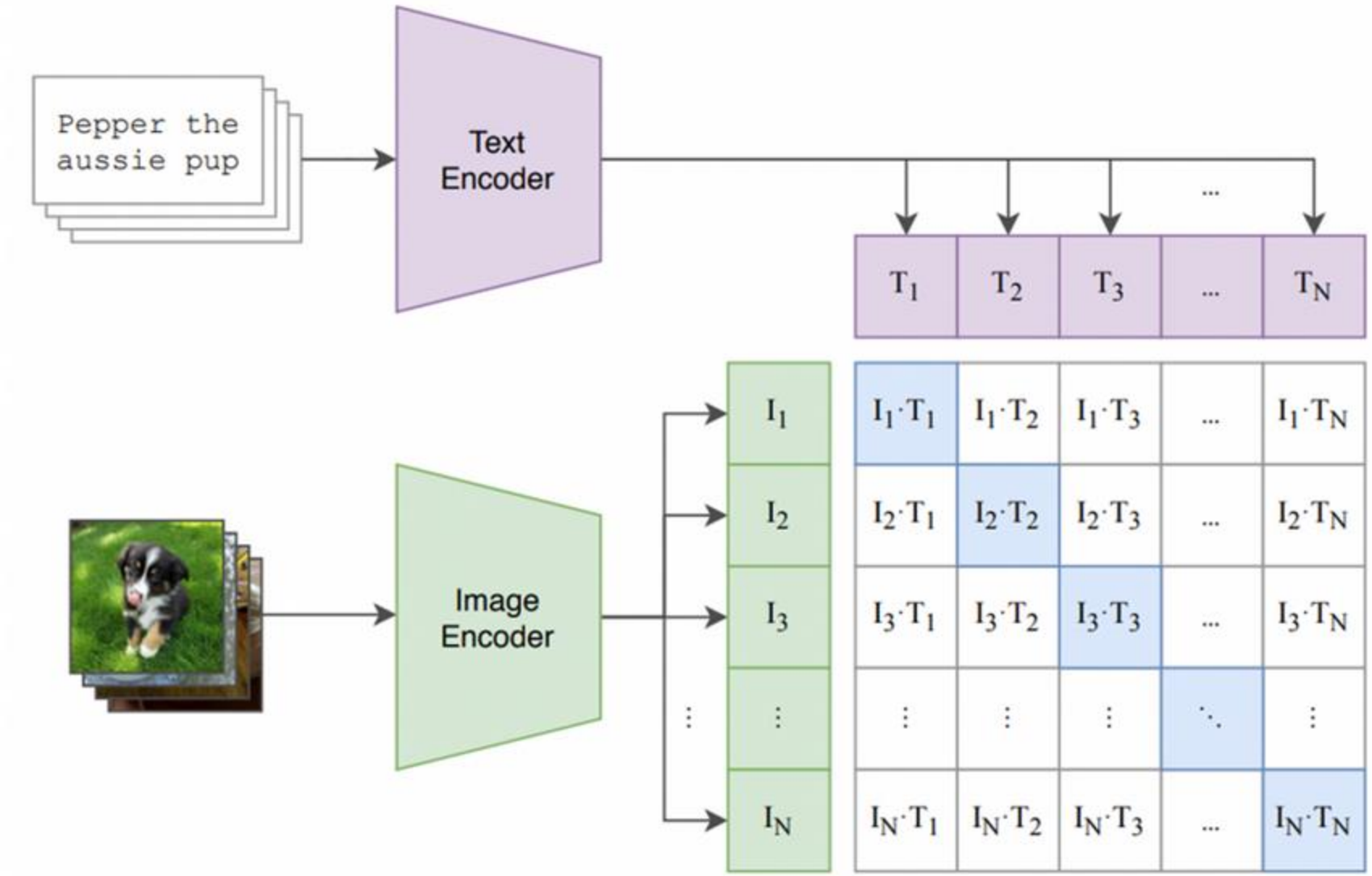
$$x_v \xrightarrow{f_v} h_v \xrightarrow{g} \tilde{h}_v \xrightarrow{\text{LLM}} y.$$

Here g might be:

- a linear projector,
- an MLP,
- a resampler,
- a query transformer.

- **BLIP-2** uses a frozen image encoder, a frozen LLM, and a lightweight Q-Former to bridge them.
- **Flamingo** uses a pretrained vision encoder and language model connected through gated cross-attention layers, allowing interleaved image-text inputs.
- **LLaVA** uses a CLIP-style visual encoder plus a projection into an LLM, then instruction tuning.
- **Qwen2-VL / Qwen2.5-VL** are strong recent open multimodal models with broad image understanding and document-style capabilities.

CLIP



Learning Transferable Visual Models From Natural Language Supervision

Data curation for CLIP

- WebImageText (WIT) OpenAI Private Dataset
  - ➢ 400M image-text data pairs
  - ➢ search for (image, text) pairs as part of the construction process whose text includes one of a set of 500,000 queries
  - ➢ balance the results by including up to 20,000 (image, text) pairs per query

Demystifying CLIP Data (Xu et al. ICLR 2024)

- **Dual Encoder Setup:**

$$\mathcal{F}_{\text{img}} : \mathcal{X} \to \mathbb{R}^d, \quad \mathcal{F}_{\text{text}} : \mathcal{T} \to \mathbb{R}^d.$$

  CLIP learns separate encoders for images ($\mathcal{F}_{\text{img}}$) and text ($\mathcal{F}_{\text{text}}$).

- **Contrastive Training Objective:**

$$\mathcal{L} = -\sum_{(i,t)} \left[\log \frac{\exp\big(\text{sim}\big(\mathcal{F}_{\text{img}}(i), \mathcal{F}_{\text{text}}(t)\big)/\tau\big)}{\sum_{t'} \exp\big(\text{sim}(\mathcal{F}_{\text{img}}(i), \mathcal{F}_{\text{text}}(t'))/\tau\big)} + \log \frac{\exp\big(\text{sim}\big(\mathcal{F}_{\text{text}}(t), \mathcal{F}_{\text{img}}(i)\big)/\tau\big)}{\sum_{i'} \exp\big(\text{sim}(\mathcal{F}_{\text{text}}(t), \mathcal{F}_{\text{img}}(i'))/\tau\big)} \right],$$

  where $\text{sim}(\cdot, \cdot)$ is a similarity function (e.g. cosine similarity) and $\tau > 0$ is a temperature parameter.

- **Inference via Similarity:** For an image i and a set of text prompts $\{t_1, \ldots, t_k\}$, compute

$$s_j = \text{sim}\big(\mathcal{F}_{\text{img}}(i), \mathcal{F}_{\text{text}}(t_j)\big).$$

  The predicted label is $\arg\max_j \ s_j$.

- **Few/Zero-Shot Classification:** By crafting textual prompts (e.g. "`A photo of a [class].`") and comparing similarities, CLIP can classify new categories with minimal or no additional training data.

Domain-specific prompts

- ‘‘A cartoon-style depiction of a [*class*]’’
- ‘‘A pencil-sketch image of a [*class*]’’
- ‘‘A low-resolution photograph of a [*class*]’’
- ‘‘A highly stylized painting of a [*class*]’’
- ‘‘A thermal image of a [*class*]’’

- **Handcrafted Bias:** Manually engineered prompts assume prior knowledge of domain styles (e.g.“cartoon,” “sketch”) and may fail when the domain is unseen.
- **Scaling Issue:** Crafting/tuning prompts for many domains quickly becomes cumbersome and is not guaranteed to capture all variations.
- **Overfitting Risk:** Overly tailored prompts can overfit to known source domains, hurting performance on truly novel domains.

Prompt engineering to prompt learning

Motivation: Prompt learning aims to *jointly* optimize textual *prompt embeddings* so that a vision-language model (e.g. CLIP) becomes more effective on downstream classification tasks.

Context Optimization (CoOp):

- Learns a small set of *context tokens* (prompt embeddings) $\{\mathbf{v}_1, \ldots, \mathbf{v}_m\}$ instead of using a fixed handcrafted prompt.
- For a class name c, the textual prompt is

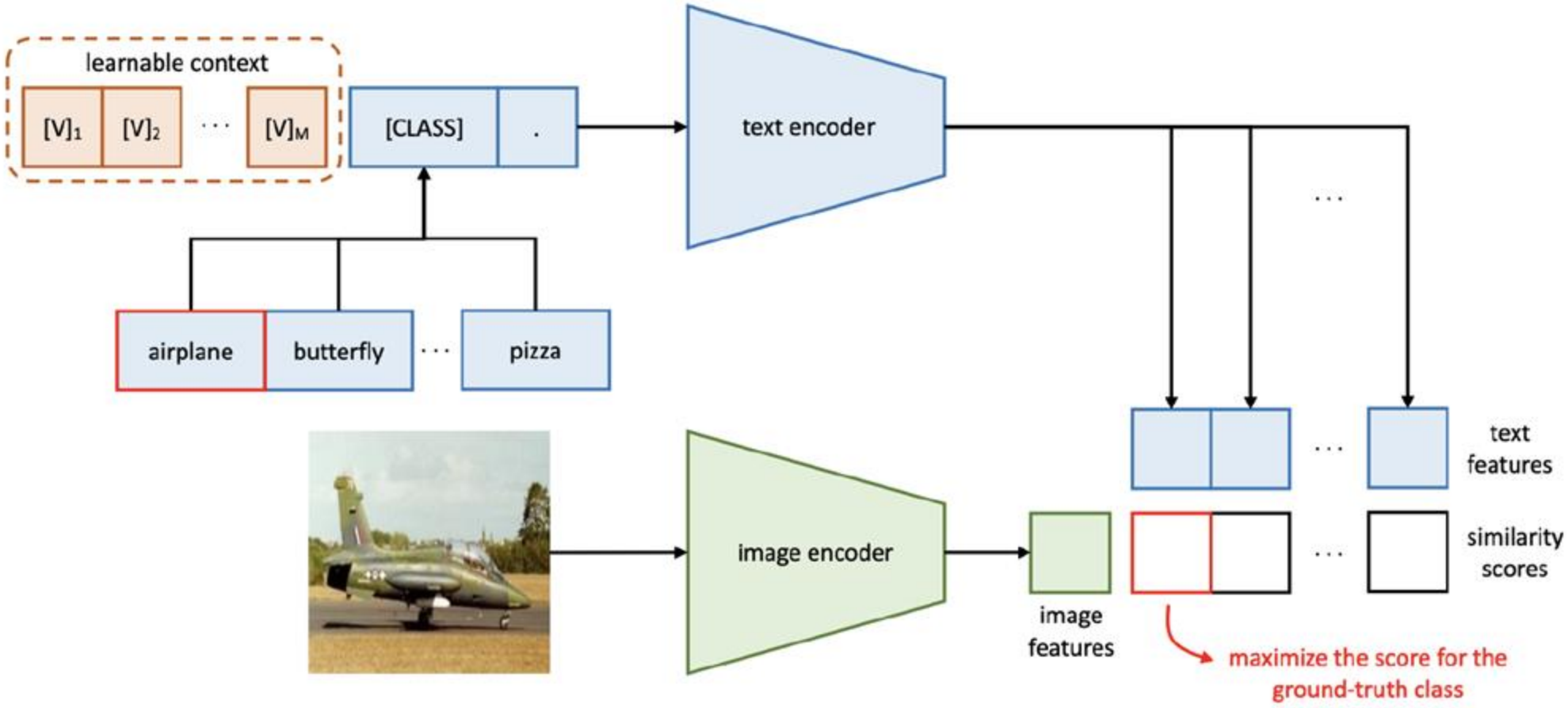
$$T(\theta) = \left[\mathbf{v}_1, \ldots, \mathbf{v}_m,\ c\right],$$

  where $\theta = \{\mathbf{v}_i\}_{i=1}^{m}$ are learnable parameters.
- Classification is done by comparing $\mathcal{F}_{\text{img}}(x)$ with $\mathcal{F}_{\text{text}}(T(\theta))$ via a similarity measure.

Conditional Context Optimization (CoCoOp):

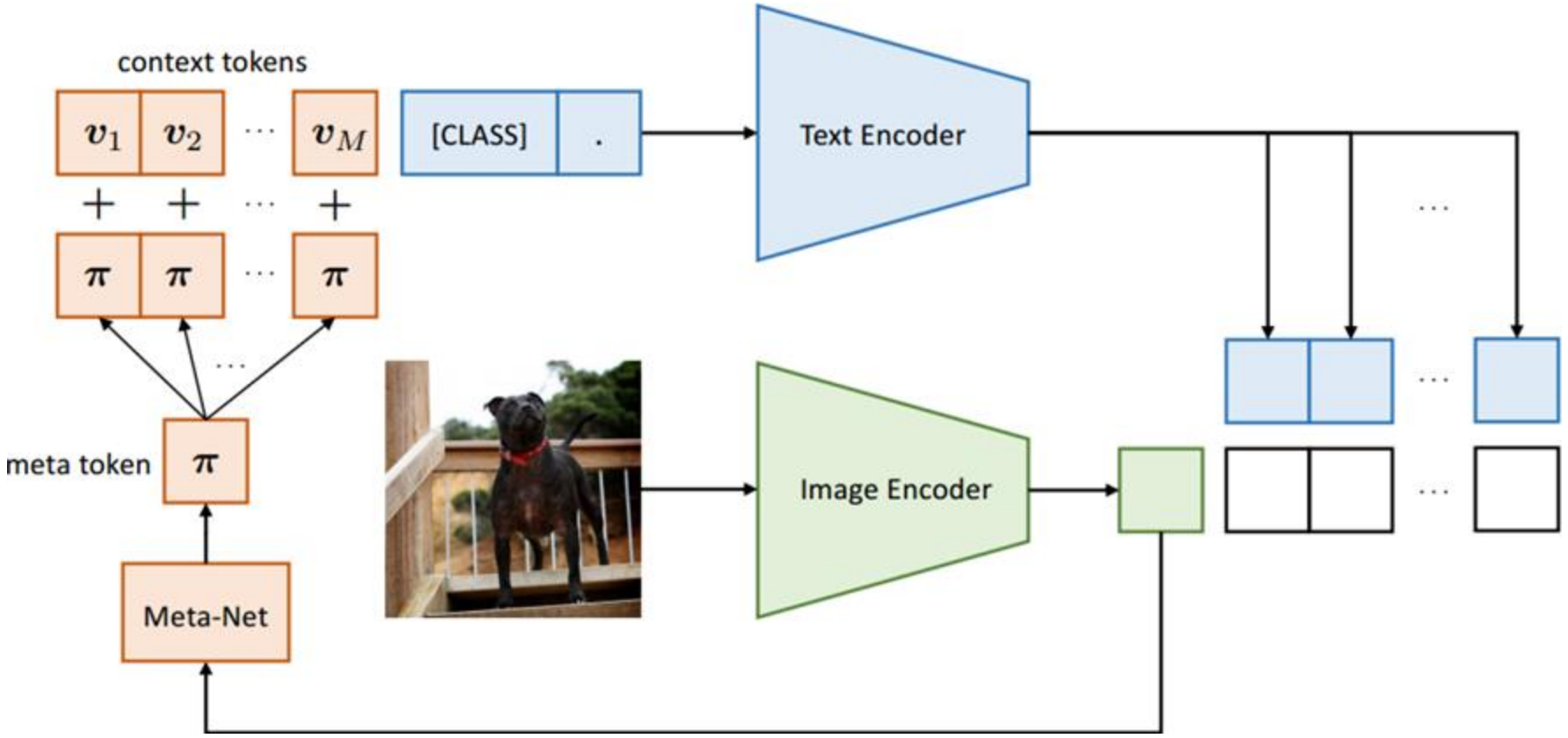
- Extends CoOp by making the context *conditional* on each input image.
- A lightweight network generates context tokens $\mathbf{v}_i(x)$ based on $\mathcal{F}_{\text{img}}(x)$, allowing prompts to adapt for different image domains or styles.
- Improves generalization by dynamically adjusting prompts to match each test instance better.

Context optimization (CoOP)



Learning to Prompt for Vision-Language Models

Conditional context optimization (CoCoOp)



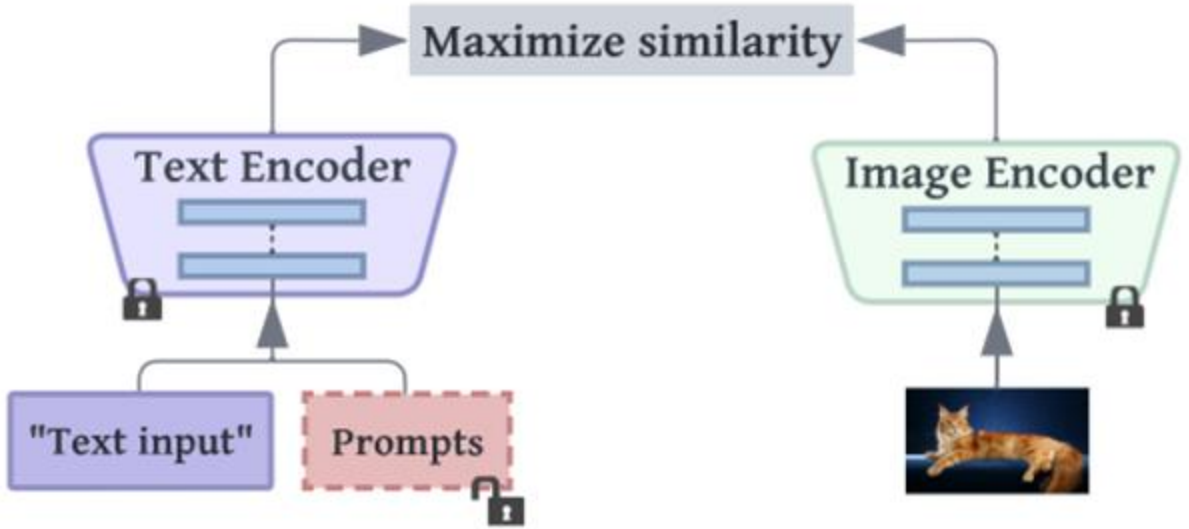
Conditional Prompt Learning for Vision-Language Models

Uni-Modal vs Multi-Modal Prompting:

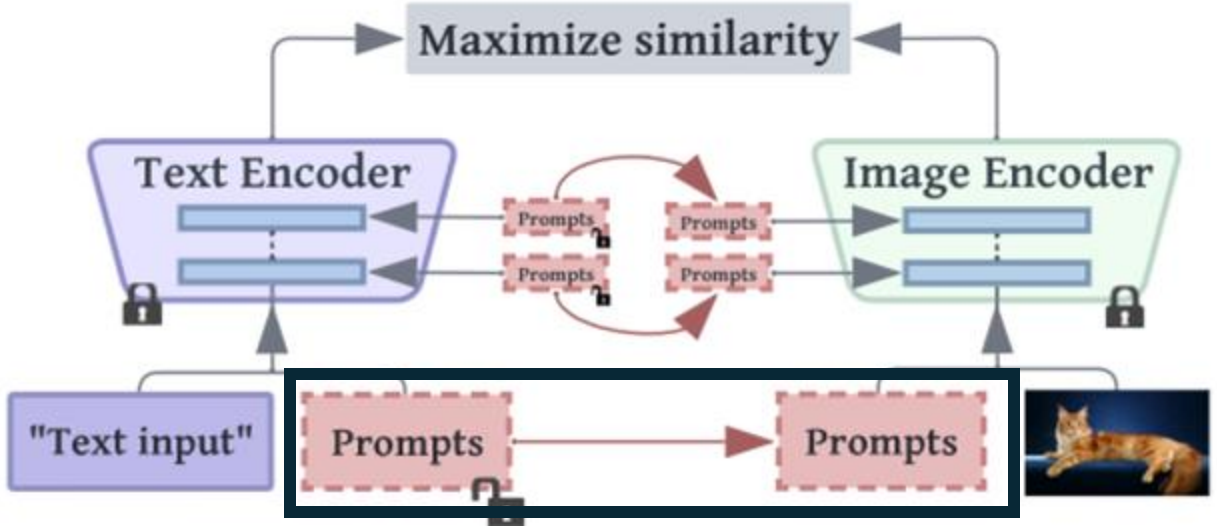
- **Enhanced Expressiveness:** Combining text and vision prompts provides richer, more nuanced contextual cues than single-modal approaches.
- **Cross-Modality Alignment:** Integrating features from both modalities improves semantic consistency and mutual adaptation.
- **Dynamic Adaptation:** Flexible conditioning allows the prompt to adjust based on complementary signals from text and images.
- **Few-/Zero-Shot Generalization:** Synergistic multimodal representations facilitate robust inference on unseen tasks with minimal labeled data.

MaPLe:

- Extends prompt learning to both vision and language branches of models like CLIP.
- Learns adaptive prompt tokens conditioned on features from $\mathcal{F}_{\text{img}}(\cdot)$ and $\mathcal{F}_{\text{text}}(\cdot)$.
- Synchronizes cross-modal prompts to yield coherent representations and robust classification performance.



(a) Existing prompt tuning methods (Uni-modal)



MaPLe: Multi-modal Prompt Learning

CLIP on distributions shift



ImageNetV2 (Recht et al. 2019)





ImageNet Sketch (Wang et al. 2019)



ObjectNet (Barbu et al. 2019)



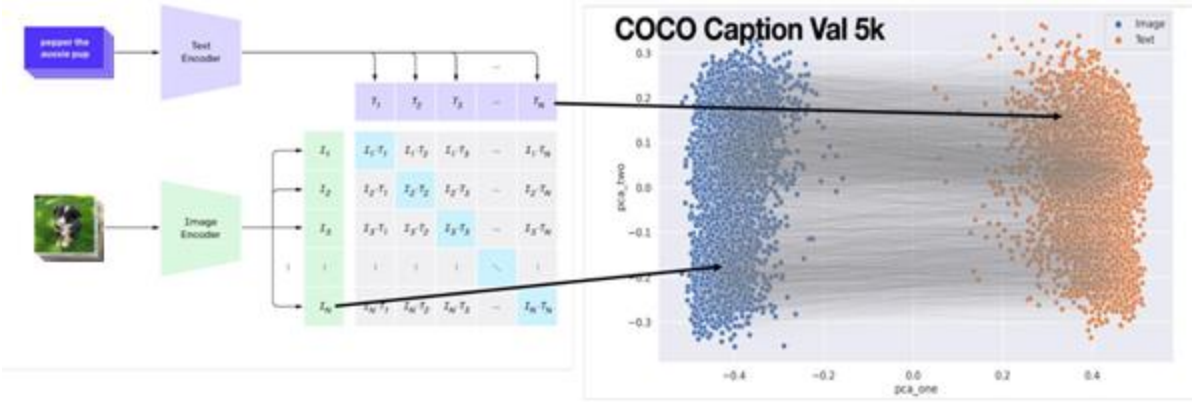
ImageNet-R (Hendrycks et al. 2021)



ImageNet-A (Hendrycks et al. 2019)

Modality Gap in CLIP

CLIP learns two coordinated coordinate systems inside the same hypersphere.



For example, an image may contain:

> a dog, grass, sunlight, motion blur, background trees, leash, shadow

but the caption may only say:

> "a dog running"

- **Different information content:** an image contains fine visual details that text may not mention
- **Different abstraction levels:** text may summarize only the salient concept, while the image contains background, style, pose, clutter, etc.
- **Encoder mismatch:** the vision encoder and text encoder learn through different inductive biases and architectures
- **Training objective usually enforces proximity, not exact equality:** contrastive losses like in CLIP mainly push matched pairs closer than unmatched pairs

Interpretability and disentanglement of CLIP

- **CLIP features are semantically meaningful but not fully disentangled:** they capture objects, attributes, background, style, and domain information together in a distributed embedding space.
- **They are interpretable through language prompts:** an image embedding can be compared with text prompts such as “has stripes”, “is a medical image”, or “contains water” to estimate semantic relevance.
- **A concept bottleneck layer maps CLIP embeddings to human-understandable concepts:** instead of directly using the raw CLIP feature, the model predicts concept scores such as color, shape, texture, object parts, or scene type.

$$x \rightarrow \text{CLIP image encoder} \rightarrow z \rightarrow \text{classifier}$$

$$x \rightarrow \text{CLIP image encoder} \rightarrow z \rightarrow \text{concept layer} \rightarrow c \rightarrow \text{classifier}$$

- **This improves transparency and control:** predictions can be explained using explicit concepts, and spurious cues such as background or style can be identified and reduced.

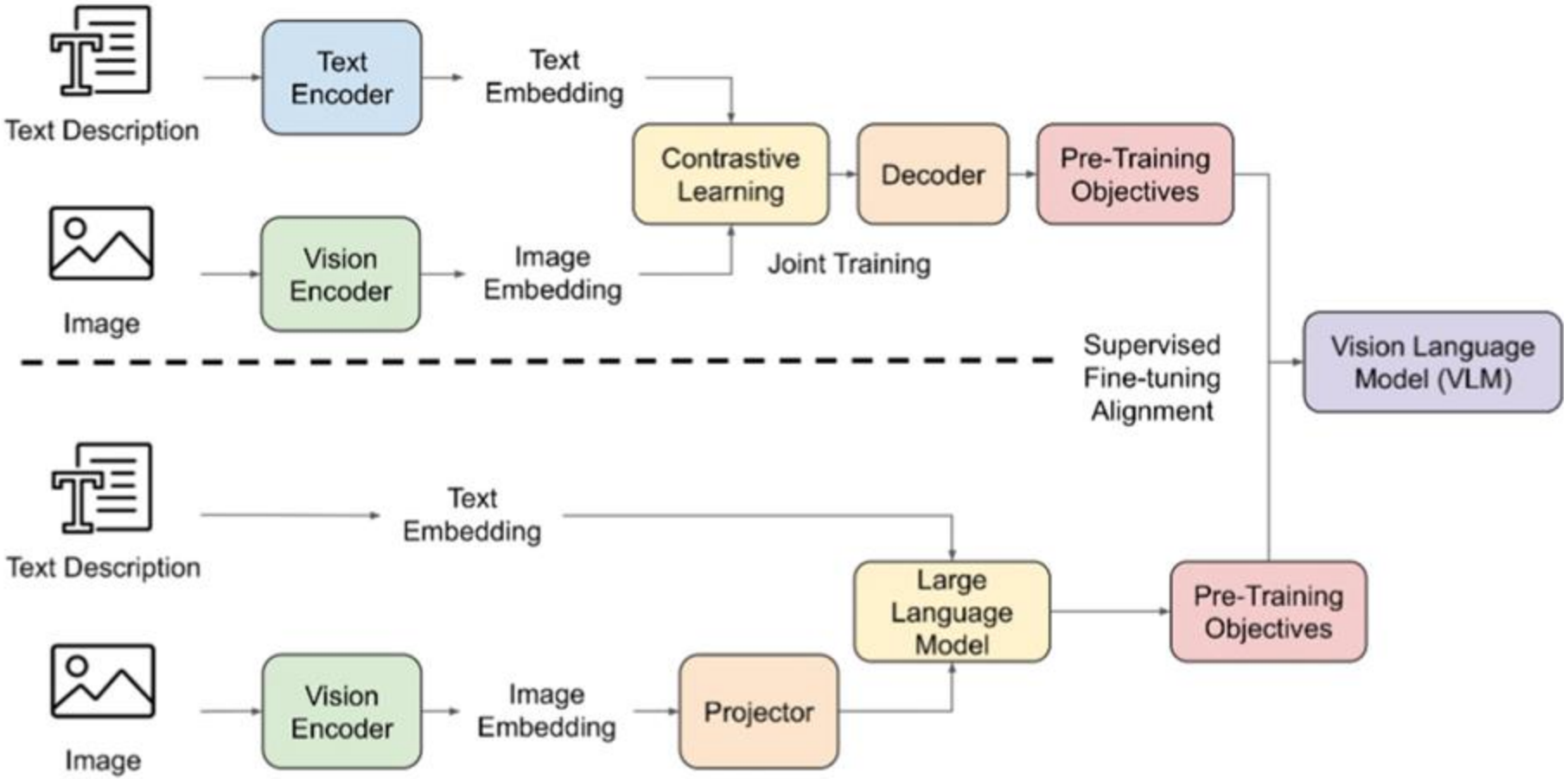
SigLIP

- SigLIP = CLIP-style vision-language pretraining with sigmoid pairwise loss instead of softmax contrastive loss
- CLIP compares each pair against the whole batch
- SigLIP scores each pair more directly as match vs non-match
- This can work better for smaller batches and scale efficiently

$$\mathcal{L}_{CLIP} = -\log \frac{\exp(s_{ii}/\tau)}{\sum_j \exp(s_{ij}/\tau)}$$

$$\mathcal{L}_{SigLIP} = -\sum_{i,j} \log \sigma(y_{ij} s_{ij})$$

Llava – vision + language to language



Visual Instruction Tuning

Architecture

- **Vision encoder:** LLaVA first uses a pre-trained vision model (commonly CLIP) to convert the input image into a set of visual feature embeddings.
- **Projection / connector module:** A small learned projector maps these visual embeddings into the embedding space of the language model so that the image information can be treated like input tokens.
- **Large language model:** The projected visual tokens are fed along with the text prompt into an LLM (such as Vicuna), which then generates the final response in a conversational, instruction-following manner.

What are the roles of the projector

- **Modality bridge**: The projector translates visual features from the vision encoder into the token embedding space of the language model so the LLM can process image information meaningfully.
- **Feature alignment**: It learns how to align image representations with language representations, making the visual content compatible with the linguistic reasoning abilities of the LLM.
- **Information packaging**: It compresses and reformats the visual signal into a form that can be injected as pseudo-tokens, allowing the language model to use image cues during response generation.

Projector design

- In **original LLaVA**, the projector is very simple: it is just a **single trainable linear layer**. The paper says LLaVA uses “a simple linear layer” and “a trainable projection matrix” to convert CLIP visual features into language embedding tokens.

$$H_v = X_v W$$

where:

- X_v = visual features from the vision encoder
- W = learned projection matrix
- H_v = projected visual tokens for the LLM

Training strategy for Llava

- **Stage 1 — Feature alignment:** The image is passed through a **frozen vision encoder**, and its visual features are mapped into the language-model embedding space using the **projector**. In this stage, **only the projector is trained**, so the goal is just to make visual features compatible with the LLM's token space. The original LLaVA paper describes this as pre-training for feature alignment using a filtered subset of CC3M.
- **Stage 2 — Visual instruction tuning:** After alignment, the model is trained on **image + instruction + answer** examples so it learns to behave like a visual assistant. Here, the **vision encoder stays frozen**, while the **projector and the LLM are updated**, allowing the model to answer questions, describe images, and perform image-grounded reasoning.
- **Purpose of the two stages:** The first stage teaches the model **how to connect image features to language**, while the second stage teaches it **how to follow multimodal instructions**. In other words, stage 1 builds the bridge; stage 2 teaches conversational and reasoning behavior on top of that bridge.

Stage-1 loss objective

- The stage-1 loss is to **maximize the likelihood of the target caption tokens given the image and the instruction**, using the same objective as a causal language model.

$$p_\theta(\mathbf{X}_a \mid \mathbf{X}_v, \mathbf{X}_q) = \prod_{i=1}^{L} p_\theta(x_{a,i} \mid \mathbf{X}_v, \mathbf{X}_q, x_{a,<i})$$

- where $\mathbf{X}_v$ is the image-derived visual tokens, $\mathbf{X}_q$ is the instruction, and $\mathbf{X}_a$ is the assistant answer. In stage 1, $\mathbf{X}_a$ is just the caption.

Visual instruction tuning

- **Goal of visual instruction tuning:** After feature alignment, LLaVA is fine-tuned on **image + instruction + response** triples so that it learns to behave like a **multimodal assistant**, not just an image-captioning model. In this stage, the **vision encoder remains frozen**, while the **projector and the language model are updated**.
- **Example instructions:** Typical prompts include: **"Describe this image in detail," "What is the person holding?", "How many animals are visible?", "What might happen next in this scene?", "Why is this situation potentially dangerous?"** These examples reflect the kinds of synthetic image-grounded instructions Llava generates using GPT-4 from captions and object information.

$$\mathcal{L}_{\text{VIT}} = -\sum_{i=1}^{L} \log p_\theta(x_{a,i} \mid X_v, X_q, x_{a,<i})$$

Q-former based connector (BLIP type models)

Q-Former is a lightweight Transformer module that converts a large set of visual patch tokens into a small set of learnable query-based visual tokens that can be passed to a language model.

$$\mathbf{V} = [v_1, v_2, \ldots, v_N] \in \mathbb{R}^{N \times d_v}$$

where $\mathbf{V}$ denotes frozen vision-encoder features, e.g., from CLIP/ViT.

The Q-Former introduces K learnable query tokens:

$$\mathbf{Q}^{(0)} = [q_1, q_2, \ldots, q_K] \in \mathbb{R}^{K \times d_q}, \qquad K \ll N$$

At each layer, the query tokens interact among themselves through self-attention and extract image information through cross-attention:

$$\tilde{\mathbf{Q}}^{(\ell)} = \mathrm{SelfAttn}\left(\mathbf{Q}^{(\ell-1)}\right)$$

$$\mathbf{Q}^{(\ell)} = \mathrm{CrossAttn}\left(\tilde{\mathbf{Q}}^{(\ell)}, \mathbf{V}, \mathbf{V}\right)$$

where the queries attend to visual tokens as:

$$\mathrm{CrossAttn}(\mathbf{Q}, \mathbf{V}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{W}_Q(\mathbf{V}\mathbf{W}_K)^\top}{\sqrt{d}}\right)\mathbf{V}\mathbf{W}_V$$

After L layers, Q-Former produces compact visual tokens:

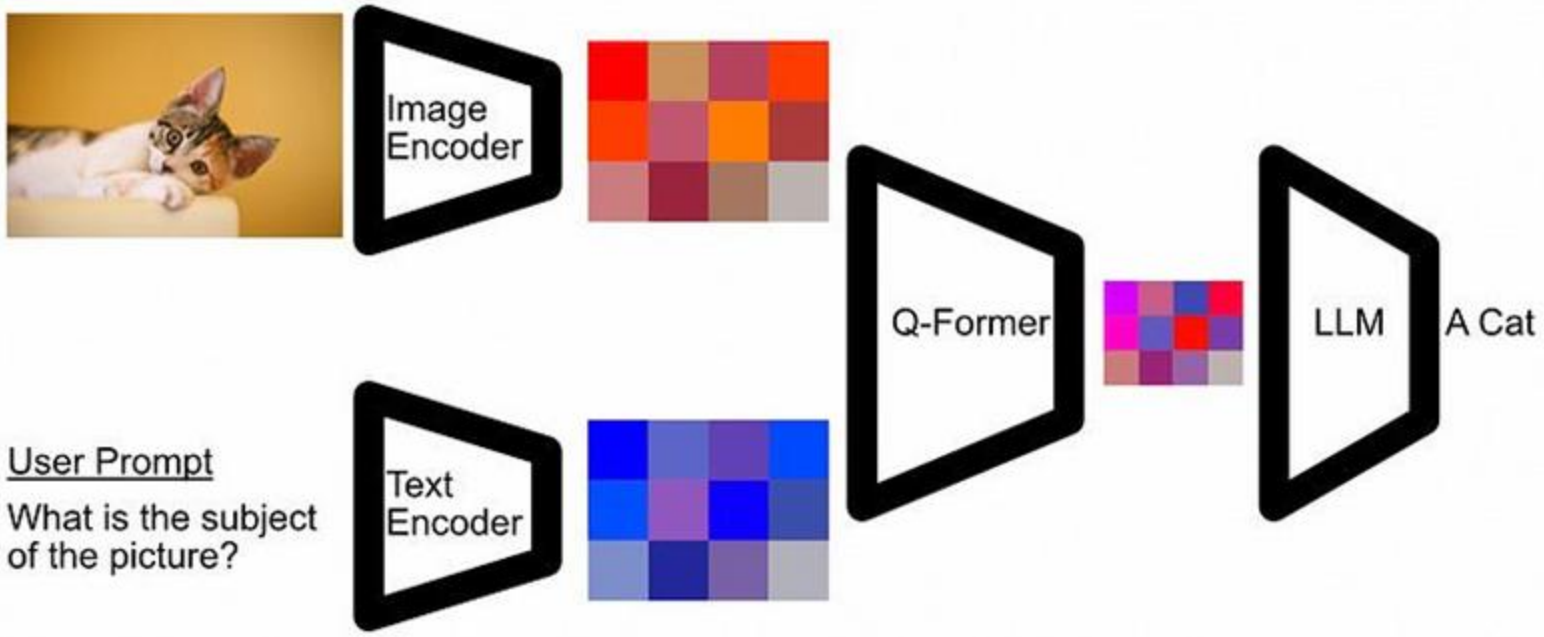
$$\mathbf{Z}_Q = \mathbf{Q}^{(L)} \in \mathbb{R}^{K \times d_q}$$

These are projected into the LLM embedding space:

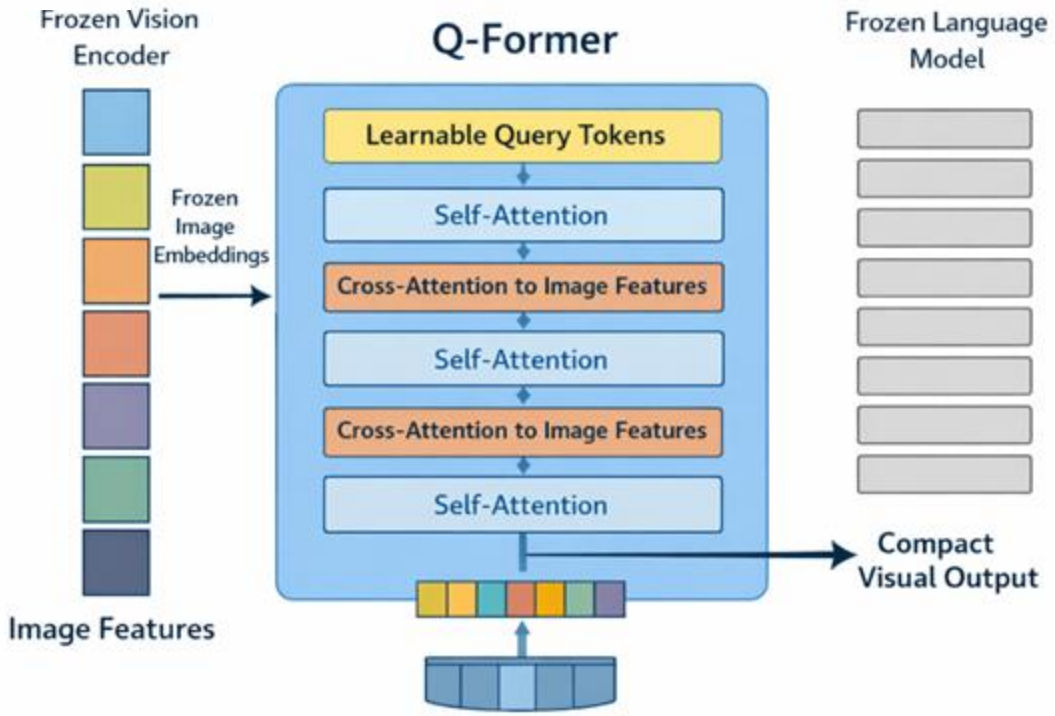
$$\mathbf{H}_{\text{vis}} = \mathbf{Z}_Q \mathbf{W}_p$$

and used as visual context for language generation:

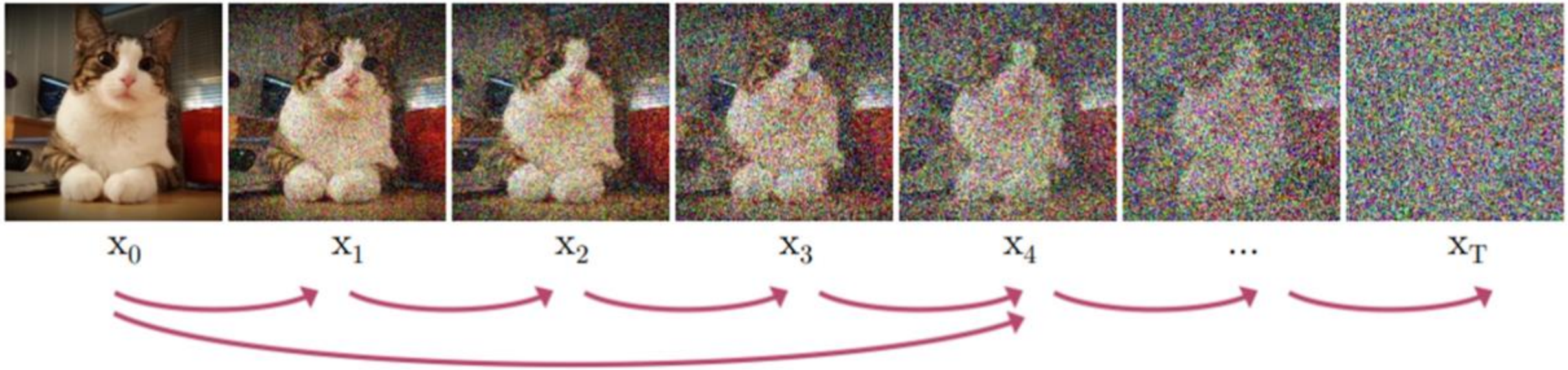
$$p(y_t \mid y_{<t}, \mathbf{H}_{\text{vis}}, x_{\text{text}})$$



Q-former in BLIP



Forward process



Markov Property

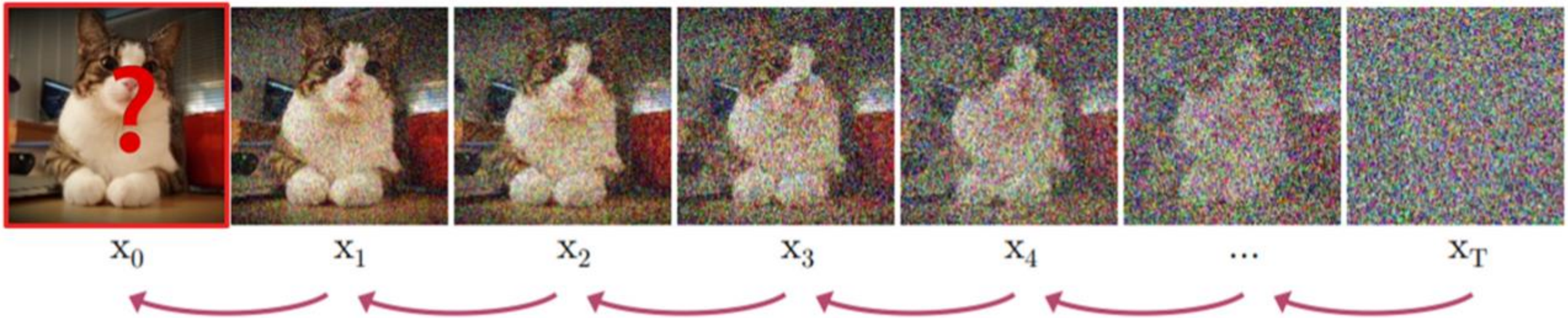
$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$$ ← **Diffusion Kernel**

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\,\epsilon \qquad \text{where} \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\alpha_t := 1 - \beta_t \text{ and } \bar{\alpha}_t := \prod_{s=0}^{t} \alpha_s$$

Reverse process



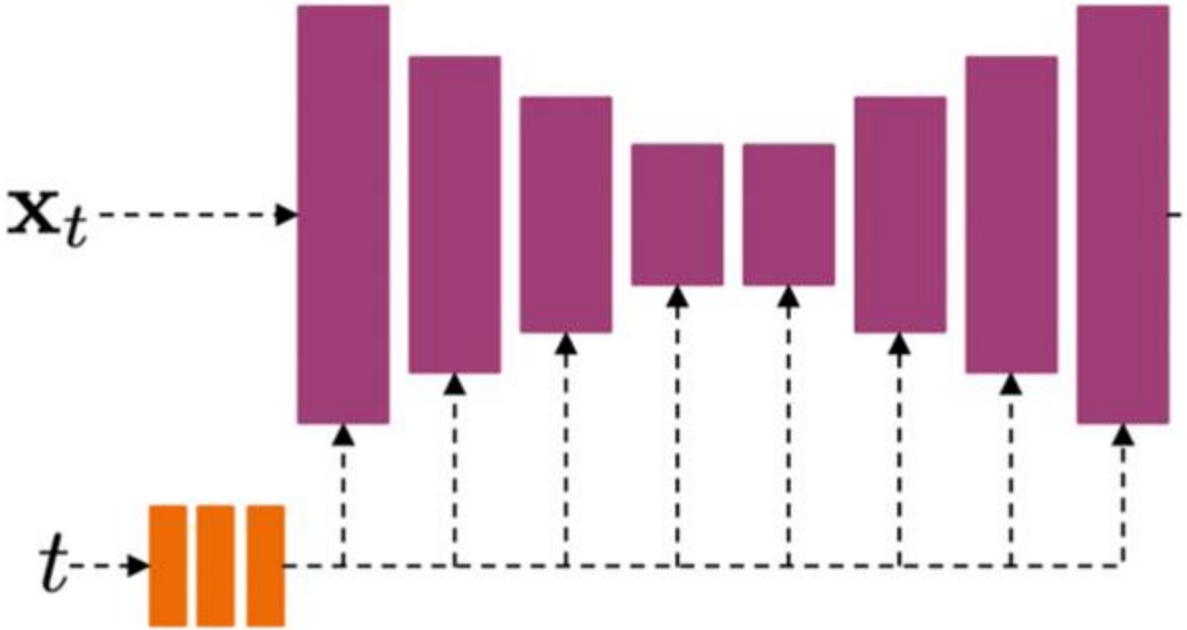
$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$$

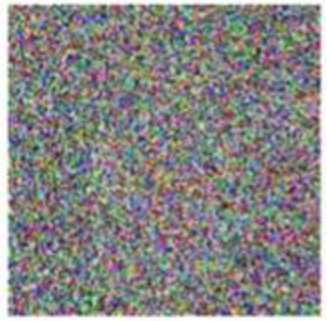
Model

$$p_\theta(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

Denoising Diffusion - Training







Variational Lower Bound Loss

$$L = \mathbb{E}_q\Big[D_{\mathrm{KL}}(q(x_T \mid x_0) \,\|\, p(x_T)) + \sum_{t=2}^{T} D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)) - \log p_\theta(x_0 \mid x_1)\Big]$$

Simpler formulation

$$L_{\text{simple}} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right] \qquad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

BalancedDPO: Adaptive Multi-Metric Alignment

Dipesh Tamboli[†*] Souradip Chakraborty[‡] Aditya Malusare[†] Biplab Banerjee[§]
Amrit Singh Bedi[¶] Vaneet Aggarwal[†]

[†] Purdue University [‡] University of Maryland
[§] Indian Institute of Technology Bombay [¶] University of Central Florida

- **DPO learns from winner–loser pairs:** for the same prompt c, the model is trained to assign higher likelihood to the preferred image x^w than the rejected image x^l.
- **No explicit reward model is needed:** DPO directly optimizes the generator using preference pairs, instead of first learning a separate reward function.
- **Reference model prevents drift:** the updated model is compared against the original pretrained model p , so alignment does not destroy generation quality.
- **It is usually fine-tuning, not retraining:** in diffusion models, DPO typically updates the denoising network or lightweight adapters such as LoRA, rather than training the whole model from scratch.
-

Instead of combining several reward values directly, BalancedDPO asks each reward model a much simpler question:

> Given two images for the same prompt, which one do you prefer?

Each reward model casts a **binary vote**. For reward model k, the vote is:

$$s_k(c, x_0^1, x_0^2) = \begin{cases} 1 & \text{if } r_k(c, x_0^1) \geq r_k(c, x_0^2) \\ -1 & \text{otherwise} \end{cases}$$

So if CLIP prefers image 1, that is a $+1$. If Aesthetic prefers image 2, that becomes a -1, and so on. Then the method simply takes the **majority vote** across all metrics:

$$s(c, x_0^1, x_0^2) = \text{sign}\left(\sum_{k=1}^{K} s_k\right)$$

Convert multiple heterogeneous rewards into one consensus preference signal

Some future directions

- **Stronger visual grounding and faithfulness**: future VLMs will be pushed to tie every generated claim to image regions, objects, and evidence, because grounding is now a central open problem behind hallucination and unreliable reasoning.
- **Tool-using and agentic VLMs**: instead of answering from one forward pass alone, VLMs are increasingly being designed to call OCR, detection, segmentation, retrieval, calculators, and other tools during reasoning.
- **Beyond image-text to video, 3D, robotics, and action**: the field is moving from static image understanding toward embodied and temporal intelligence, including video reasoning and vision-language-action models.
- **More efficient and adaptable VLMs**: a major trend is parameter-efficient tuning, lightweight multimodal connectors, and deployment-friendly models for edge or resource-constrained settings.
- **Domain-specialized VLMs**: rather than one generic web model for everything, there is growing interest in VLMs tailored to domains such as remote sensing, healthcare, and scientific analysis, where grounding and domain shift matter much more.

Future VLMs will be more grounded, tool-using, multimodal, efficient, and domain-aware.